ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

MÔN HỌC: HỆ ĐIỀU HÀNH (CS333)

LAB 2
SYSTEM CALLS

Sinh viên thực hiện:	Võ Nhất Bảo	24120166
	Phạm Lê Huy Hoàng	24120182
Mã nhóm:	24120166_24120182	

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Thông tin nhóm và phân công công việc 2
2 Tổng quan và môi trường thực hiện 2
3 Task 3.1 – Using gdb 3
4 Task 3.2 – Sandbox a command 4
5 Task 3.3 – Sandbox with allowed pathnames 6
6 Task 3.4 – Attack xv6 7
7 Kiểm thử 8
8 Thảo luận và tự đánh giá 9
9 Tài liệu tham khảo 10

1. Thông tin nhóm và phân công công việc

STT	MSSV	Họ và tên	Công việc đảm nhận	Đóng góp
1	24120166	Võ Nhất Bảo	Task 3.1: chạy gdb với QEMU, trả lời sáu câu hỏi (`answers-syscall.txt`), thu thập ảnh minh chứng. Task 3.2: system call `interpose`, chặn system call theo mask, kế thừa mask qua `fork`. Phân tích đường đi của trang nhớ cho Task 3.4.	50%
2	24120182	Phạm Lê Huy Hoàng	Task 3.3: cho phép `open`/`exec` theo pathname. Task 3.4: tấn công xv6 (`user/attack.c`). Viết test bổ sung (`user/sbaudit.c`, `test-sandbox-audit.py`), kiểm chứng độc lập trên môi trường thứ hai.	50%
Công việc chung			Chạy autograder, rà soát chéo mã nguồn, `time.txt`, viết báo cáo, đóng gói bài nộp.	Cả hai

Bảng 1. Phân công công việc và tỷ lệ đóng góp của các thành viên.

Task	Nội dung	Tệp chính	Hoàn thành	Autograder
3.1	Using gdb	`answers-syscall.txt`	100%	5/5
3.2	Sandbox theo mask	`kernel/{syscall,sysproc,proc}.c`, `kernel/proc.h`, `user/usys.pl`, `user/user.h`	100%	20/20
3.3	Sandbox theo pathname	`kernel/{syscall,sysproc,proc}.c`, `kernel/proc.h`	100%	20/20
3.4	Attack xv6	`user/attack.c`	100%	14/14
–	Số giờ thực hiện	`time.txt`	100%	1/1
Tổng kết quả `make grade`				**60/60**

Bảng 2. Mức hoàn thành do nhóm tự đánh giá và điểm script `make grade`. Với Task 3.1, script chỉ kiểm tra `answers-syscall.txt` có nội dung; sáu câu trả lời được chấm thủ công. Các test thuộc từng task được liệt kê ở mục 7.

2. Tổng quan và môi trường thực hiện

Lab 2 đi vào bên trong kernel xv6: dùng GDB để theo dõi một system call và lần ra nguyên nhân của một kernel panic (Task 3.1), thêm system call mới `interpose` để "sandbox" một process (Task 3.2, 3.3), và khai thác một lỗi rò rỉ bộ nhớ được cố ý đưa vào kernel (Task 3.4).

Môi trường. Giống Lab 1: NixOS 26.05 (x86-64), `shell.nix` cung cấp `riscv64-none-elf-gcc` 15.2.0, Binutils 2.46, QEMU 10.2.4 (3 CPU), GDB 17.2, Make 4.4.1, Python 3.13. `shell.nix` và `scripts/capture_lab2.py` là công cụ nội bộ, nằm ngoài repo xv6 nên không có trong `lab.zip`. Mã nguồn: `git://g.csail.mit.edu/xv6-labs-2026`, nhánh `syscall`, commit gốc `99161db`, `LAB=syscall`. **Mã bài làm được kiểm thử: commit `e969443`**; các commit sau chỉ sửa báo cáo và `docs/`. Ngày 25/10, commit này được chấm lại với GCC 15.3.0 (nixpkgs đã cập nhật), vẫn 60/60; Q4 lặp lại trên bản chưa sửa kernel (`0ab52a6`) vẫn cho `0x80001d7a`. Kiểm chứng độc lập trên WSL Ubuntu 24.04 (GCC 13.3.0, QEMU 8.2.2, GDB 15.1) cũng đạt 60/60.

Quy trình. Nhánh `syscall` được tạo từ `origin/syscall` và đẩy lên GitHub để hai thành viên làm tiếp trên cùng nhánh. Mỗi task được build và chạy thử bằng `make qemu`, chấm riêng bằng `./grade-lab-syscall <tên test>` rồi commit; hai tệp chấm điểm `grade-lab-syscall` và `gradelib.py` được giữ nguyên.

Lỗi cố ý của lab. Makefile biên dịch kernel với `-DLAB_SYSCALL`. Khi macro này được định nghĩa, `uvmalloc()` bỏ lệnh `memset(mem, 0, sz)` (`kernel/vm.c`, dòng 268–270), còn `kfree()`/`kalloc()` bỏ việc ghi rác lên trang (`kernel/kalloc.c`, dòng 57–60 và 85–88). Hệ quả: trang nhớ cấp cho process mới vẫn giữ nội dung của process trước [1][2]. Đây là cơ sở của Task 3.4, và cũng ảnh hưởng tới kiểm thử thủ công (mục 8).

3. Task 3.1 – Using gdb

Mở hai terminal: `make qemu-gdb` khởi động QEMU với `-S -gdb tcp::26000`, nên CPU dừng ngay từ lệnh đầu tiên và chờ debugger; `gdb` đọc `.gdbinit` để kết nối (`target remote`) và nạp symbol từ `kernel/kernel`. Mọi số liệu dưới đây lấy từ phiên chạy thật ngày 20/10 trên nhánh `syscall` *trước* khi sửa kernel (GCC 15.2.0); địa chỉ phụ thuộc bản build. Ảnh chụp do script nội bộ `scripts/capture_lab2.py` (ngoài gói nộp) tạo: script gõ lệnh vào gdb thật trong xterm rồi chụp cửa sổ. Câu trả lời đầy đủ (tiếng Anh) nằm trong `answers-syscall.txt`. Cơ chế trap và system call theo chương 2, mục 4.3–4.4 của sách xv6 [3]; ý nghĩa các bit của `sstatus`, `scause` theo đặc tả RISC-V privileged [4].

```
(gdb) b syscall
Breakpoint 1 at 0x80001d6a: file kernel/syscall.c, line 138.
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
138     {
(gdb) backtrace
#0  syscall () at kernel/syscall.c:138
#1  0x0000000080001b2a in usertrap () at kernel/trap.c:68
#2  0x0000003ffffff09c in ?? ()
(gdb) n
140       struct proc *p = myproc();
(gdb) n
142       num = p->trapframe->a7;
(gdb) p /x *p
$1 = {lock = {locked = 0x0, name = 0x80007130, cpu = 0x0}, state = 0x4, chan = 0x0, killed = 0x0, xstate = 0x0,
  pid = 0x1, parent = 0x0, kstack = 0x3fffffd000, sz = 0x4000, pagetable = 0x87f52000, trapframe = 0x87f56000,
  context = {ra = 0x8000135c, sp = 0x3fffffdc60, s0 = 0x3fffffdc90, s1 = 0x80007cf0, s2 = 0x800078c0, s3 = 0x0,
    s4 = 0x3, s5 = 0x80018990, s6 = 0x0, s7 = 0x18c, s8 = 0xffffffffffffffff, s9 = 0x400, s10 = 0x2, s11 = 0x38},
  ofile = {0x0 <repeats 16 times>}, cwd = 0x80015e00, name = {0x69, 0x6e, 0x69, 0x74, 0x0 <repeats 12 times>}}
(gdb) p p->trapframe->a7
$2 = 15
(gdb) p /x $sstatus
$3 = 0x200000022
(gdb)
```

Hình 1. Breakpoint tại `syscall`: `backtrace`, nội dung `struct proc`, `trapframe->a7` và `sstatus` (Q1–Q3).

Q1. Hàm nào đã gọi `syscall`? Theo frame `#1` của `backtrace`, đó là `usertrap()` tại `kernel/trap.c:68`. Frame `#2` ở địa chỉ `0x3ffffff09c` = `TRAMPOLINE + 0x9c`, chính là nhãn `userret`: `uservec` gọi `usertrap()` bằng lệnh `jalr t0`, nên địa chỉ trả về là lệnh ngay sau nó. GDB hiện `??` vì mã trampoline được link ở `0x80006000` (symbol `uservec`), còn khi chạy nó được map ở địa chỉ ảo `TRAMPOLINE = MAXVA - PGSIZE = 0x3ffffff000`.

Q2. Giá trị của `p->trapframe->a7` là gì và nó biểu diễn điều gì? `a7 = 15`, tức `SYS_open` trong `kernel/syscall.h`. Stub trong `user/usys.S` nạp số hiệu system call vào `a7` rồi gọi `ecall`; `uservec` lưu các thanh ghi user vào trapframe, và `syscall()` đọc lại `a7` từ đó để chọn hàm xử lý.

Gợi ý trong đề CS333 [1] (xem `user/initcode.S`, nơi giá trị là `SYS_exec`) dựa trên bản xv6 cũ; trang MIT 2026 [2] đã đổi gợi ý thành `user/init.c` và `user/init.asm`. Bản 2026 không còn `initcode.S`: `forkret()` (`kernel/proc.c`) gọi trực tiếp `kexec("/init")` trong kernel cho process đầu tiên. Vì vậy system call đầu tiên đi qua `syscall()` là `open("console", O_RDWR)` ở đầu `main` của `user/init.c`. Kết quả `p /x *p` khẳng định điều này: `pid = 0x1`, `name = {0x69, 0x6e, 0x69, 0x74}` = `"init"`, `state = 0x4` (RUNNING).

Q3. CPU đang ở mode nào trước đó? User mode. `p /x $sstatus` cho `0x200000022`. Bit `SPP` (bit 8) lưu mode trước khi trap: 0 là user, 1 là supervisor. Các bit khác cũng khớp với đường đi của system call (Hình 2).

63–34	33–32	31–9	8	7–6	5	4–2	1	0
0	UXL = 10_2	0	SPP = 0	0	SPIE = 1	0	SIE = 1	0

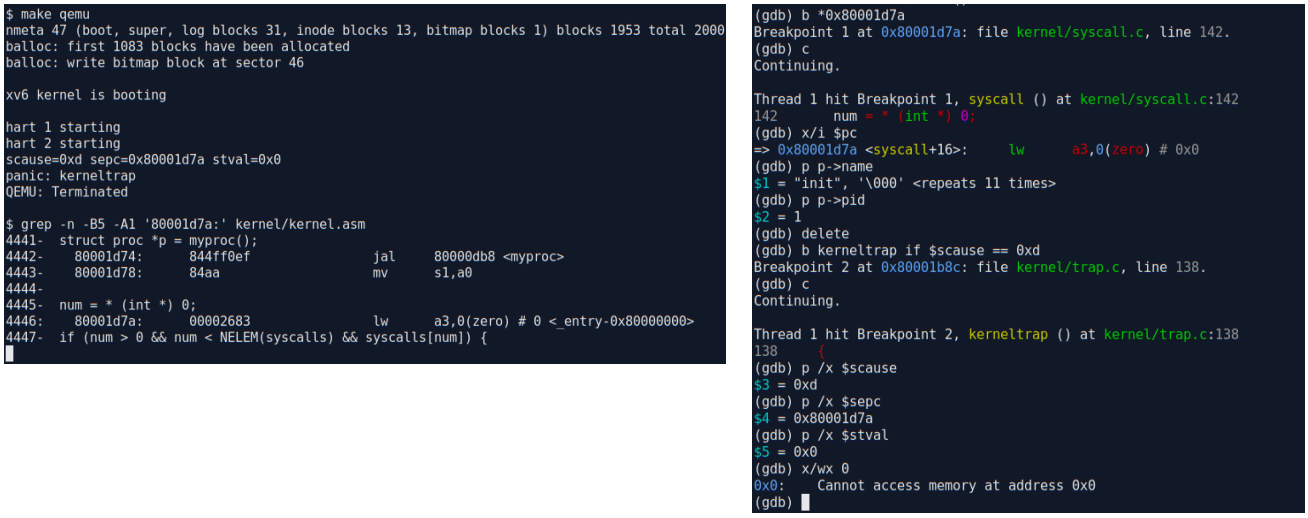
`SPP = 0`: trap đến từ user mode
`SPIE = 1`: trước trap, bit `SIE` đang bật
`SIE = 1`: `usertrap()` gọi `intr_on()` trước `syscall()`
`UXL = 2`: user mode dùng thanh ghi 64 bit

Hình 2. Giải mã `sstatus = 0x200000022`.

Q4. Lệnh assembly nào gây panic? Thanh ghi nào ứng với biến `num`? Thay `num = p->trapframe->a7;` bằng `num = *(int *) 0;` rồi chạy `make qemu`, kernel in `scause=0xd sepc=0x80001d7a stval=0x0` và `panic: kerneltrap`. Tìm `80001d7a` trong `kernel/kernel.asm` (Hình 3, trái) được lệnh

```
80001d7a:  00002683    lw  a3,0(zero)
```

tức nạp 4 byte từ địa chỉ `zero + 0` = `0` vào thanh ghi `a3`. Vậy `num` nằm trong **`a3`**; lệnh kế tiếp `addiw a4,a3,-1` dùng `a3` để kiểm tra `num > 0 && num < NELEM(syscalls)`. Trong gdb, breakpoint `b *0x80001d7a` dừng đúng tại dòng 142 của `syscall.c` và `x/i $pc` hiện cùng lệnh (Hình 3, phải).



Hình 3. Trái: panic khi đọc địa chỉ 0 và lệnh tương ứng trong `kernel.asm`. Phải: gdb tại lệnh lỗi và tại `kerneltrap` (Q4–Q6).

Q5. Vì sao kernel bị crash?

Kernel xv6 chạy với phân trang bật, dùng kernel page table do `kvmmake()` (`kernel/vm.c`) tạo. Bảng này chỉ map các vùng ở bảng bên, khớp Hình 3-3 của sách xv6 [3]; địa chỉ ảo `0` không thuộc vùng nào. Khi lệnh `lw` đọc địa chỉ 0, MMU không tìm được PTE hợp lệ và sinh exception:

- `scause = 0xd` = 13: *load page fault* (bit ngắt = 0, mã exception 13 theo đặc tả RISC-V privileged [4]);
- `stval = 0x0`: địa chỉ ảo gây lỗi;
- `sepc = 0x80001d7a`: lệnh `lw` gây lỗi.

Trap đi vào `kernelvec` rồi `kerneltrap()`. Đây không phải ngắt thiết bị và kernel không bao giờ được phép page fault, nên `kerneltrap()` in ba thanh ghi trên rồi gọi `panic("kerneltrap")`. GDB xác nhận bằng breakpoint có điều kiện `b kerneltrap if $scause == 0xd`: `$scause = 0xd`, `$sepc = 0x80001d7a`, `$stval = 0x0`. Lệnh `x/wx 0` cũng không đọc được địa chỉ 0.

Vùng	Địa chỉ ảo	Quyền
PLIC	`0x0c000000–0x10000000`	RW
UART0	`0x10000000` (1 trang)	RW
VIRTIO0	`0x10001000` (1 trang)	RW
Mã kernel	`0x80000000–etext`	RX
Dữ liệu + RAM	`etext–0x88000000`	RW
Kernel stack	dưới `TRAMPOLINE`	RW
Trampoline	`0x3ffffff000`	RX

Bảng 3. Các vùng được map trong kernel page table; `0x0` nằm ngoài tất cả.

> Cùng địa chỉ 0 lại *hợp lệ* trong user page table, vì mã của chương trình user bắt đầu ở địa chỉ ảo 0. Kernel page table không map bộ nhớ user, nên kernel phải đọc dữ liệu user qua `copyin/fetchaddr`.

Q6. Process nào đang chạy khi kernel panic? PID là bao nhiêu? Tại breakpoint `0x80001d7a`: `p p->name` = `"init"`, `p p->pid` = 1. Đó là process **`init`** (PID 1), đang thực hiện system call đầu tiên, nên shell chưa kịp khởi động. Sau khi trả lời, câu lệnh `num = p->trapframe->a7;` được khôi phục (script tự khôi phục và `git status` sạch).

4. Task 3.2 – Sandbox a command

Yêu cầu. Thêm system call `int interpose(int mask, const char *path)`. Nếu bit `n` của `mask` được bật thì system call số `n` của process gọi *không được thực thi* và trả về -1. Process con tạo bằng `fork` thừa hưởng hạn chế của cha. Ở task này `path` luôn là `"-"` (được dùng ở Task 3.3).

4.1. Các tệp đã sửa và trường mới trong `struct proc` (Task 3.2–3.3)

Tệp	Thay đổi
`Makefile`	Thêm `$U/_sandbox`, `$U/_sbtest` vào `UPROGS`; `$U/_sbaudit` (test bổ sung) chỉ được thêm khi build với `SANDBOX_AUDIT=1`.
`kernel/syscall.h`, `user/usys.pl`, `user/user.h`	`#define SYS_interpose 23`; `entry("interpose")` sinh stub trong `user/usys.S`; khai báo `int interpose(int, const char *)`.
`kernel/syscall.c`	Thêm `[SYS_interpose]` vào bảng `syscalls[]`; hàm `sandbox_rejects()` kiểm tra mask (3.2) và pathname của `open/exec` (3.3), được gọi một lần trong `syscall()`.
`kernel/sysproc.c`	`sys_interpose()`: OR mask mới vào mask cũ (3.2); đọc và kiểm tra pathname, không cho nới ngoại lệ đã có (3.3).
`kernel/proc.h`, `kernel/proc.c`	Hai trường ở Bảng 5; `kfork()` sao chép cho con; `freeproc()` xóa khi thu hồi slot.

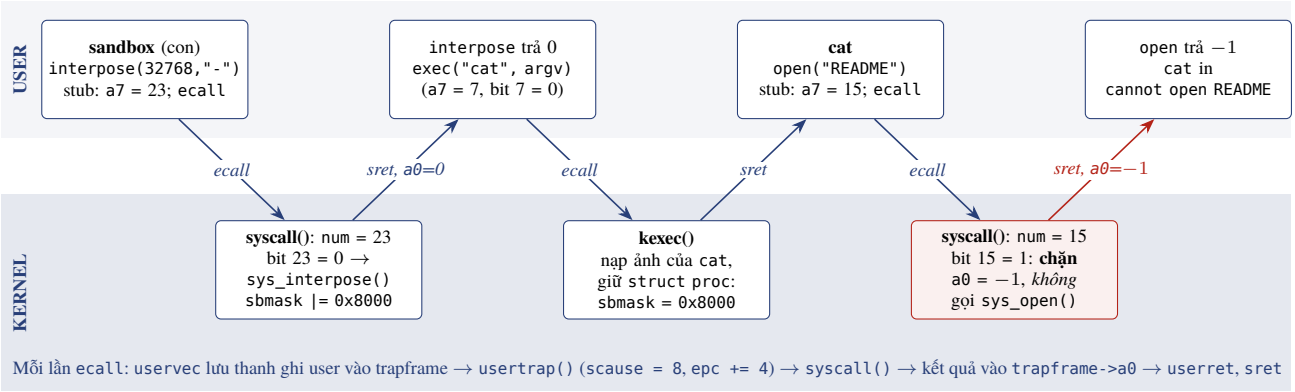
Bảng 4. Các tệp được sửa trong Task 3.2 và 3.3.

Trường	Kiểu	Ý nghĩa	Vòng đời
`sbmask`	`uint`	Bit n = 1 thì system call số n bị từ chối. Kiểu không dấu để bit 31 cũng được xử lý đúng.	Chỉ được OR thêm bit trong `sys_interpose()`; `kfork()` sao chép; `kexec()` giữ nguyên; `freeproc()` đặt 0.
`sbpath`	`char[MAXPATH]`	Pathname duy nhất mà `open`/`exec` bị mask vẫn được dùng; `"-"` nghĩa là không có ngoại lệ (3.3).	Ghi trong `sys_interpose()` khi pathname hợp lệ và không nới quyền; `kfork()` sao chép; `kexec()` giữ nguyên; `freeproc()` xóa.

Bảng 5. Các trường mới trong `struct proc` (`kernel/proc.h`).

4.2. Đường đi của lời gọi `interpose` và của một system call bị chặn

Hình 4 theo dõi lệnh `sandbox 32768 - cat README`. `sandbox` fork một process con; process con gọi `interpose(32768, "-")` rồi `exec` chương trình `cat`. $32768 = 2^{15}$ = `1 << SYS_open`.



Hình 4. `interpose` ghi mask vào `struct proc`; mask sống qua `exec` và chặn `open` của `cat` ngay tại dispatcher.

4.3. Thiết kế và cài đặt

- **Kiểm tra một lần, trước khi gọi handler.** `syscall()` gọi `sandbox_rejects()` sau khi `num` đã được xác nhận nằm trong bảng ($1 \le$ `num` ≤ 23, nên phép dịch `1U << num` luôn hợp lệ). System call bị chặn không chạy handler nên không có tác dụng phụ: `fork` bị chặn không tạo process con. Cách này không phải sửa từng handler, và chặn được cả system call không có đối số như `getpid`, `fork` (test `sandbox_block`).
- **Hạn chế chỉ được cộng dồn.** `sys_interpose()` dùng `|=` chứ không gán. Test `sandbox_escape` chạy `sbtest escape` dưới mask `open`; chương trình gọi `interpose(0, 0)` và `interpose(0, "README")` để thử thoát. Nếu dùng phép gán, mask 0 xóa luôn bit 15 và process tự thoát sandbox.
- **Kế thừa qua `fork`, giữ qua `exec`.** `kfork()` sao chép mask cho process con. `kexec()` thay ảnh bộ nhớ nhưng giữ `struct proc`, nên mask đặt trước `exec` vẫn còn hiệu lực. Nhờ vậy `sandbox 32768 - sh < exec.sh` chặn được cả `ls` do `sh` fork ra (test `sandbox_fork`).
- **Khởi tạo đúng cho process mới.** `freeproc()` xóa mask cùng chỗ với `pid`, `killed`, `xstate`. `allocproc()` chỉ cấp slot `UNUSED`, tức slot đã qua `freeproc()` hoặc mảng `proc[]` tĩnh vốn bằng 0, nên process không tạo bằng fork không thể nhận mask cũ.
- **Đồng bộ.** Hai trường chỉ được process tự đọc/ghi, được `kfork()` ghi trước khi process con thành `RUNNABLE`, và được `freeproc()` ghi khi process đã kết thúc. Vì vậy chúng thuộc nhóm "private to the process" như `sz`, `pagetable`, không cần giữ `p->lock`.

kernel/syscall.c – kiểm tra sandbox tại dispatcher

```
160   num = p->trapframe->a7;
161   if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
162     if (sandbox_rejects(p, num)) {
163       // Fail without running the handler.
164       p->trapframe->a0 = -1;
165     } else {
166       // Use num to lookup the system call function for num, call it,
167       // and store its return value in p->trapframe->a0
168       p->trapframe->a0 = syscalls[num]();
169     }
```

kernel/proc.c – freeproc() và kfork()

```
170   p->sbmask = 0;
171   p->sbpath[0] = 0;
```

```
294   // the child stays in the parent's sandbox.
295   np->sbmask = p->sbmask;
296   safestrcpy(np->sbpath, p->sbpath, sizeof(np->
          sbpath));
```

Bản cuối của `sandbox_rejects()` và `sys_interpose()` (đã gồm phần pathname của Task 3.3) được trình bày ở mục 5.

4.4. Chạy thử

```
init: starting sh
$ sandbox 32768 - sh < exec.sh
$ "hello world!"
$ ls: cannot open .
$ $ echo SCRIPT_DONE
SCRIPT_DONE
$ sandbox 32768 - cat README
cat: cannot open README
$ sandbox 32768 - echo "hello"
"hello"
$ sandbox 128 - grep xv6 README
sandbox: exec grep failed
$ sandbox 32768 - sbtest getpid
sbtest: getpid ret 15
sbtest: alive
$ sandbox 2048 - sbtest getpid
sbtest: getpid ret -1
sbtest: alive
$ sandbox 2 - sbtest fork
sbtest: fork ret -1
sbtest: alive
$ sandbox 32768 - sbtest escape
sbtest: escape confined
sbtest: alive
$ sbtest getpid
sbtest: getpid ret 22
sbtest: alive
$
```

Hình 5. Các mask khác nhau trong xv6.

Mask	Bit	Kết quả
32768	`open` (15)	`sh < exec.sh`: `echo` vẫn in `"hello world!"`, `ls` báo `cannot open .`; `cat: cannot open README`; `echo` không mở tệp nên chạy bình thường.
128	`exec` (7)	`sandbox: exec grep failed`: chính process con của `sandbox` không exec được.
32768	`open` (15)	`sbtest getpid` trả PID thật (15): system call không bị mask vẫn chạy.
2048	`getpid` (11)	`getpid ret -1`, process vẫn `alive`.
2	`fork` (1)	`fork ret -1`, không có dòng `CHILD RAN`: handler không chạy.
32768	`open` (15)	`sbtest escape`: `escape confined`.
–	–	`sbtest getpid` chạy sau cùng, ngoài sandbox, trả PID 22: slot process tái sử dụng không giữ mask cũ.

5. Task 3.3 – Sandbox with allowed pathnames

Yêu cầu. Mở rộng `interpose(mask, path)`: một lời gọi `open` hoặc `exec` đã bị mask vẫn được phép nếu pathname của nó *bằng đúng* `path`. Mọi system call bị mask khác, và `open`/`exec` với pathname khác, vẫn bị chặn như Task 3.2. `path = "-"` nghĩa là không cho phép pathname nào.

- **Lưu pathname an toàn.** `sys_interpose()` đọc `path` bằng `argstr(1, path, MAXPATH)` vào buffer cục bộ *trước* khi đổi trạng thái. Con trỏ sai, chuỗi không có NUL hoặc dài từ `MAXPATH` = 128 byte trở lên đều trả -1, mask và `sbpath` giữ nguyên. Kernel giữ bản sao riêng, nên user sửa buffer của mình sau đó không ảnh hưởng tới chính sách.
- **Kiểm tra tại dispatcher.** `sandbox_rejects()` cho chạy mọi call không bị mask. Với call bị mask, chỉ `open`/`exec` có ngoại lệ: `sbpath` khác `"-"`, đối số 0 đọc được bằng `argstr`, và trùng *chính xác* `sbpath`. So sánh chuỗi không chuẩn hóa đường dẫn: `README` khác `./README`; `---` là tên tệp thường, chỉ đúng chuỗi `"-"` mới mang nghĩa "không có ngoại lệ" (test `sandbox_minus`).
- **Không nới quyền.** Khi `open` hoặc `exec` đã bị mask, lời gọi mới chỉ được chấp nhận nếu `path` giống `sbpath` hiện tại hoặc là `"-"` (bỏ ngoại lệ). Đây là lý do `sbtest escape` vẫn bị giam: `interpose(0, "README")` bị từ chối. Riêng `interpose(0, 0)` *không* làm `argstr` thất bại. Trong `sbtest`, địa chỉ ảo 0 là lệnh đầu tiên của `.text` (byte `01 11 06 ec 22 e8 00`…), nên `argstr` đọc được một chuỗi 6 byte kết thúc trong giới hạn; lệnh in tạm trong `sys_interpose()` ghi nhận `argstr = 6` và `sbmask = 0x8000`. Chuỗi này khác `"-"` và `sbpath`, nên chính quy tắc không nới quyền trả -1. Vì vậy không thể coi `interpose(0, 0)` là phép thử con trỏ sai; `sbaudit` thử ca đó bằng địa chỉ ngoài không gian user và chuỗi không kết thúc trong `MAXPATH`.
- **Kế thừa.** `kfork()` sao chép `sbpath`, `kexec()` giữ nguyên, `freeproc()` xóa (mục 4.3). Test `sandbox_forkpath`: process con của `sbtest` vẫn mở được `README` nhưng không mở được `exec.sh`.
- **Đọc pathname hai lần.** `sandbox_rejects()` và handler (`sys_open`, `sys_exec`) mỗi bên đọc pathname một lần. Giữa hai lần đọc không có mã user nào chạy, và process xv6 không chia sẻ bộ nhớ, nên chuỗi không thể bị đổi ở giữa.

kernel/syscall.c – sandbox_rejects()

```
139 static int
140 sandbox_rejects(struct proc *p, int num)
141 {
142   char path[MAXPATH];
143
144   if ((p->sbmask & (1U << num)) == 0)
145     return 0;
146   if ((num == SYS_open || num == SYS_exec) &&
147       strncmp(p->sbpath, "-", MAXPATH) != 0 &&
148       argstr(0, path, sizeof(path)) >= 0 &&
149       strncmp(path, p->sbpath, MAXPATH) == 0)
150     return 0;
151   return 1;
152 }
```

kernel/sysproc.c – sys_interpose()

```
117 uint64
118 sys_interpose(void)
119 {
120   int mask;
121   char path[MAXPATH];
122   struct proc *p = myproc();
123
124   argint(0, &mask);
125   if (argstr(1, path, sizeof(path)) < 0)
126     return -1;
127   if ((p->sbmask & ((1U << SYS_open) | (1U << SYS_exec))) &&
128       strncmp(path, p->sbpath, MAXPATH) != 0 &&
129       strncmp(path, "-", MAXPATH) != 0)
130     return -1;
131   p->sbmask |= (uint)mask;
132   safestrcpy(p->sbpath, path, sizeof(p->sbpath));
133   return 0;
134 }
```

```
init: starting sh
$ sandbox 128 sh sh < exec.sh
$ exec echo failed
$ exec ls failed
$ $ echo SCRIPT_DONE
SCRIPT_DONE
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
$ cat README > x
$ sandbox 32768 README grep xv6 x
grep: cannot open x
$ echo hello! > ---
$ sandbox 32768 --- grep hello! ---
hello!
$ sandbox 32768 README sbtest forkpath
sbtest: forkpath child README ok
sbtest: forkpath child exec.sh denied
sbtest: alive
$ sandbox 32768 - sbtest escape
sbtest: escape confined
sbtest: alive
$
```

Hình 6. Sandbox theo pathname trong xv6.

- `sandbox 128 sh sh < exec.sh`: `exec("sh")` được phép vì trùng `"sh"`; trong script, `exec` của `echo` và `ls` bị chặn.
- `sandbox 32768 README grep xv6 README`: `grep` mở được `README`, in các dòng chứa `xv6`; với tệp `x` (bản sao của `README`) thì báo `cannot open x`.
- `sandbox 32768 --- grep hello! ---`: `---` là pathname được phép, không phải ký hiệu `"-"`.
- `sbtest forkpath`: process con thừa hưởng ngoại lệ `README`; `sbtest escape`: vẫn bị giam.
- Test `sandbox_most` dùng mask 2147483643 = mọi bit 0–30 trừ bit 2 (`exit`) với ngoại lệ `grep`: chỉ `exec grep` và `exit` chạy được, nên không có dòng `exec ... failed` nào.

6. Task 3.4 – Attack xv6

Yêu cầu. `user/secret.c` ghi một chuỗi bí mật vào bộ nhớ rồi thoát. Chỉ sửa `user/attack.c` sao cho nó tìm và in ra chuỗi đó trên một dòng; `secret.c` và kernel giữ nguyên.

6.1. Secret nằm ở đâu

`secret` gọi `sbrk(9 * 4096)` (cấp phát ngay, không lazy). Sau `exec`, `sz` đã căn theo trang, nên vùng mới bắt đầu đúng đầu một trang và `align = 0`. Chương trình cộng thêm `32 + (unsigned char)secret[0]`, rồi ghi chuỗi đánh dấu `"Here it is: "` (12 byte), tiếp theo là secret và byte NUL. Vậy trong **trang đầu** của vùng `sbrk`, chuỗi đánh dấu nằm ở offset $32 + c$ và secret ở offset $44 + c$, với $c \in [0, 255]$ là mã của ký tự đầu. Ví dụ `xyzzy`: `'x'` = 120, chuỗi đánh dấu ở offset 152, secret ở 164. `secret` thoát mà không xóa gì, và do lỗi cố ý `LAB_SYSCALL`, nội dung này còn nguyên khi trang được cấp lại.

6.2. Trang nào được tái sử dụng

`kfree()` đẩy trang vào *đầu* free list, `kalloc()` lấy từ đầu: free list là một ngăn xếp (LIFO). Offset ở mục 6.1 suy ra từ `secret.c`; còn trang nào được cấp lại thì phải *quan sát* trong một phiên chạy. Nhóm đo bằng một patch tạm (chỉ trên worktree riêng, không có trong bài nộp). Patch in mỗi lần `kalloc()`/`kfree()` (thao tác, địa chỉ trang, hàm gọi, pid, tên process, cờ `SECRET` nếu trang có marker ở offset $32 + c$) và mỗi trang mà `uvmalloc()` map. **Phiên đo:** commit `e969443`, GCC 15.3.0, QEMU 10.2.4 (3 CPU); boot một lần rồi chạy `secret xyzzy`, `attack`, `attack`. Script phân tích (`addr2line` và mô phỏng free list) xác nhận mọi lần `kalloc()` đều lấy đúng trang vừa `kfree()` gần nhất; hai lần boot cho cùng chuỗi sự kiện. Patch, script, `trace.log` và `analysis.txt` nằm ở `docs/verification/nixos-20261025/` trong repo nhóm (ngoài `lab.zip`).

Bước	Sự kiện (trang vật lý `0x87f25000` chứa secret)	Trace: thao tác, pid tên: hàm gọi × số trang	Trên secret
1	`sh` gọi `wait()` thu hồi `secret`: `uvmunmap` giải phóng trang user theo địa chỉ ảo tăng dần (trang secret rồi 8 trang `sbrk` còn lại), `freewalk` giải phóng 5 trang page table.	`F 2 sh`: `freeproc`×1, `uvmunmap`×13 (secret là trang thứ 5, dòng 179), `freewalk`×5	13
2	`fork` của `sh` lấy 11 trang: trapframe, 5 trang page table và bản sao 5 trang của `sh`.	`A 2 sh`: `allocproc`×1, `uvmcreate`×1, `walk`×2, `uvmcopy`×1, `walk`×2, `uvmcopy`×4	2
3	Con của `sh` gọi `malloc` khi phân tích lệnh; `umalloc` xin tối thiểu 16 trang qua `sbrk`. Trang secret là trang heap thứ 3, không bị ghi đè vì `malloc` chỉ ghi header ở đầu heap và cấp khối từ cuối heap.	`A 4 sh`: `uvmalloc`×16 (secret là trang thứ 3, dòng 210)	–
4	`exec("attack")` lấy 10 trang (ảnh mới, trang `argv`) rồi giải phóng ảnh `sh` cũ theo địa chỉ tăng dần: trang secret, 13 trang heap sau nó, 5 trang page table, trang `argv`.	`A 4 sh`: `sys_exec`×1, `uvmcreate`×1, `walk`×2, `uvmalloc`×1, `walk`×2, `uvmalloc`×3; `F 4 attack`: `uvmunmap`×21 (secret là trang thứ 8, dòng 262), `freewalk`×5, `sys_exec`×1	19
5	`sbrk(32 * PGSIZE)` của `attack` nhận 19 trang đó trước; trang secret là trang thứ 20, tức **index 19**.	`A 4 attack`: `uvmalloc`×32; dòng 322: `KT M 17000 0x87f25000`, tức va `0x4000` + 19 × `PGSIZE`	0

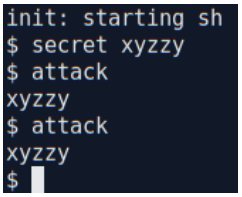
Bảng 6. Hành trình của trang chứa secret ở lần `attack` thứ nhất, theo phiên đo nêu trên. `A` = `kalloc()`, `F` = `kfree()`; số dòng chỉ vào `trace.log`. Cột cuối là số trang nằm trên trang secret trong free list sau mỗi bước.

Ở lần `attack` thứ hai (autograder chạy hai lần), trang này nằm ở index 15 (`trace.log`, dòng 511), vì lần đầu đã dùng rồi trả lại nó. Các index 19 và 15 mô tả phiên đo này, *không* phải hằng số của xv6: chúng phụ thuộc lịch sử lệnh, trạng thái heap và số trang mà `fork`, `malloc`, `exec` của `sh` lấy và trả lại. Vì vậy `attack` quét 32 trang thay vì mã hóa cứng vị trí.

6.3. Thuật toán

user/attack.c – quét các trang vừa nhận lại

```
21    for (int page = 0; page < NPAGES; page++) {
22      // secret places its marker at 32 + the first unsigned secret byte.
23      for (int off = 32; off < 32 + 256; off++) {
24        char *p = mem + page * PGSIZE + off;
25        if (memcmp(p, marker, mlen) != 0)
26          continue;
27        char *secret = p + mlen;
28        int n = 0;
29        int limit = PGSIZE - off - mlen;
30        while (n < limit && ((secret[n] >= '0' && secret[n] <= '9') ||
31                             (secret[n] >= 'A' && secret[n] <= 'Z') ||
32                             (secret[n] >= 'a' && secret[n] <= 'z')))
33          n++;
34        if (n > 0 && n < limit && secret[n] == 0 &&
35            off == 32 + (uchar)secret[0]) {
36          write(1, secret, n);
37          write(1, "\n", 1);
38          exit(0);
```



Hình 7. `attack` in đúng secret ở cả hai lần chạy.

`attack` xin 32 trang bằng `sbrk` (cấp ngay, nên các trang lấy thẳng từ free list và không bị xóa). Với mỗi trang, nó tìm chuỗi đánh dấu ở offset 32–287, đọc các ký tự chữ/số (theo đề, secret chỉ gồm chữ và số) tới NUL trong cùng trang, rồi kiểm tra `off == 32 + secret[0]`. Các điều kiện về marker, tập ký tự, NUL và offset giúp giảm nhận nhầm ứng viên, nhưng không cho biết ứng viên thuộc lần chạy `secret` nào (mục 8.2).

7. Kiểm thử

`make grade` đạt **60/60**, mã thoát 0 (Hình 8), trên cả NixOS và WSL. **60/60 là kết quả của script kiểm thử đi kèm**; điểm đồ án còn phụ thuộc nội dung trả lời GDB, báo cáo, code review và vấn đáp của từng thành viên. Mã nhóm biên dịch với `-Wall -Werror` không có cảnh báo. Cảnh báo duy nhất trong ảnh là của *linker* (`LOAD segment with RWX permissions`), do cách link kernel gốc (`kernel/kernel.ld`, nhóm không sửa).

CI trên GitHub (workflow gốc của MIT, không chạy `make grade`). Ở lần chạy gần nhất (`5e0711d`), job macOS build thành công. Job Linux dừng ở `make fmt && git diff --exit-code`: clang-format 21 đòi xóa dòng trống trong `kalloc.c`, `vm.c`, `secret.c` của mã gốc (đề cấm sửa ở Task 3.4) và căn lại `syscall.h` vì dòng `SYS_interpose`. Lỗi ở `syscall.h` đã được sửa ở `e969443`; clang-format 21.1.8 không còn đổi tệp nào nhóm sửa.

Kiểm thử bổ sung (`python3 test-sandbox-audit.py`, đạt 2/2 trên cả hai môi trường):

- `user/sbaudit.c`, 6 nhóm ca: con trỏ sai, chuỗi thiếu NUL, pathname 127/128 byte; cập nhật nguyên tử, không thay hay khôi phục ngoại lệ, cộng dồn mask; chuỗi vắt qua biên trang, kernel giữ bản sao riêng; chính sách giữ qua `exec`; `open(O_CREATE)` bị chặn không tạo tệp; bit 31 của mask. Thêm 72 lần fork (`NPROC` + 8) để kiểm tra slot process không giữ sandbox cũ.
- `attack`: 10 lần boot QEMU riêng, secret dài 1, 8, 16, 24 ký tự, bắt đầu bằng chữ số, chữ hoa hoặc chữ thường.

Task	Test	Nội dung	Điểm
3.1	`answers-syscall.txt`	Chỉ kiểm tra tệp tồn tại và có tối thiểu nội dung (`check_answers`: $\geq$ 10 ký tự); sáu câu trả lời và ảnh minh chứng được chấm thủ công theo rubric	5/5
3.2	`sandbox_mask`	Mask `open`, `exec`, cả hai	5/5
3.2	`sandbox_fork`	`sh` và process con thừa hưởng mask	5/5
3.2	`sandbox_block`	`getpid`, `fork` trả -1, không chạy handler	5/5
3.2	`sandbox_escape`	Gọi lại `interpose` không thoát được	5/5
3.3	`sandbox_path`	Ngoại lệ `README`, `x`, `sh`	5/5
3.3	`sandbox_most`	Mask gần như mọi call, chỉ cho `exec grep`	5/5
3.3	`sandbox_minus`	Pathname `---` là tên tệp thường	5/5
3.3	`sandbox_forkpath`	Process con thừa hưởng ngoại lệ	5/5
3.4	`attack`	In ra secret của lần chạy `secret` trước	14/14
–	`time`	`time.txt` chứa một số nguyên (7)	1/1
Tổng			**60/60**

Bảng 7. Các test của `grade-lab-syscall`.

```
$ make grade

make clean
/nix/store/hm3031qdg9p7rk6carbrk16xmhbl8vdp-riscv64-none-elf-binutils-2.46/bin/riscv64-none-elf-ld.bfd: warning: kerne
l/kernel has a LOAD segment with RWX permissions
```

```
== Test answers-syscall.txt ==
answers-syscall.txt: OK
== Test sandbox_mask ==
$ make qemu-gdb
sandbox_mask: OK (13.6s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (0.7s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (1.1s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (0.9s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (1.1s)
```

```
== Test sandbox_block ==
$ make qemu-gdb
sandbox_block: OK (0.9s)
== Test sandbox_escape ==
$ make qemu-gdb
sandbox_escape: OK (1.0s)
== Test sandbox_forkpath ==
$ make qemu-gdb
sandbox_forkpath: OK (0.9s)
== Test attack ==
$ make qemu-gdb
attack: OK (1.0s)
== Test time ==
time: OK
Score: 60/60

$ echo $?
0
```

Hình 8. Toàn bộ kết quả các test của `make grade` và mã thoát (chạy với `MAKEFLAGS=s` nên không in dòng lệnh build). Ảnh gốc được chia thành dải trên và hai cột.

8. Thảo luận và tự đánh giá

8.1. Khó khăn và cách khắc phục

Vấn đề	Nguyên nhân	Cách khắc phục
Đề CS333 gợi ý xem `user/initcode.S` cho Q2 nhưng tệp không tồn tại	Bản xv6-labs-2026 chạy `/init` trực tiếp bằng `kexec()` trong `forkret()`; trang MIT 2026 đã đổi gợi ý sang `user/init.c`	Đọc `forkret()` và `user/init.c`; đối chiếu `p->name = "init"`, `pid = 1`.
`sh < exec.sh` chạy sau vài lệnh khác: process con của `sh` bị page fault trong `malloc()`	Lỗi cố ý `LAB_SYSCALL`: BSS của `sh` vừa exec chứa rác, biến `freep` khác 0	Tái hiện được cả khi *không* dùng sandbox. Autograder boot lại cho mỗi test; trong demo, lệnh này chạy đầu tiên.
Không biết trước trang nào chứa secret	Index phụ thuộc số trang `fork`/`malloc`/`exec` lấy và trả lại	Quét có giới hạn 32 trang, kiểm tra offset; đo bằng patch tạm, lưu trace để giải thích (index 19 và 15 trong phiên đo).
`interpose(0, 0)` không làm `argstr` thất bại	VA 0 là mã của `sbtest`; `argstr` đọc được chuỗi 6 byte kết thúc trong `MAXPATH`	Không dựa vào lỗi của `argstr`: quy tắc "không nới quyền" từ chối mọi pathname khác `sbpath` và `"-"`; con trỏ sai được thử bằng địa chỉ ngoài user space.

Bảng 8. Các khó khăn gặp phải trong quá trình thực hiện.

8.2. Hạn chế

- Ngoại lệ là một pathname duy nhất, so khớp nguyên văn (không chuẩn hóa `./README`). Khi `open/exec` đã bị mask, không thể đổi sang ngoại lệ khác, kể cả khi hẹp hơn: lựa chọn bảo thủ để không bao giờ nới quyền.
- `attack` in ứng viên *đầu tiên* thỏa điều kiện; thuật toán không xác định tuổi của dữ liệu. Nếu trong một phiên chạy `secret` nhiều lần, bộ nhớ có thể còn secret cũ. Với chuỗi lệnh `secret xyzzy`, `attack` (in `xyzzy`), `secret 0Kq9`, `attack` (in `xyzzy`, tức secret cũ), `attack` (in `0Kq9`), lần thử đầu sau secret mới trả secret cũ và lần thử thứ hai trả đúng. Đề và autograder chấp nhận nếu một trong hai lần `attack` in ra secret [1][2], và mỗi test boot lại nên chỉ có một secret. Tuy vậy, đây vẫn là giới hạn về hành vi. Giới hạn 32 trang được chọn theo phiên đo ở mục 6.2, không phải bảo đảm cho mọi chuỗi lệnh.

8.3. Tự đánh giá theo thang điểm

Hạng mục	Điểm	Mức tự đánh giá	Minh chứng
1. Using gdb	1.5	Xuất sắc	6/6 câu, mỗi câu có output gdb/ảnh chụp thật.
2a. `interpose` plumbing	0.5	Xuất sắc	Đủ prototype, stub, số hiệu, bảng `syscalls[]`, handler; build `-Werror` không cảnh báo.
2b. Mask rejection	1.25	Xuất sắc	`sandbox_mask`, `sandbox_block` đạt; kiểm tra một lần trong `syscall()`, handler không chạy.
2c. Inheritance by `fork`	0.75	Xuất sắc	`sandbox_fork` đạt; `kfork()` sao chép, `freeproc()` xóa; 72 lần fork không rò sandbox.
3a. Allowed pathname stored	0.75	Xuất sắc	`argstr` vào buffer `MAXPATH`, kiểm tra lỗi và độ dài, kế thừa qua `fork`, xử lý `"-"`.
3b. `open/exec` path check	1.75	Xuất sắc	`sandbox_path`, `sandbox_most`, `sandbox_minus` đạt; `sandbox_mask`, `sandbox_fork` vẫn đạt.
4. Attack xv6	2.0	Xuất sắc	`attack` đạt, 10/10 lần boot riêng; chỉ sửa `user/attack.c`; giải thích offset (từ source) và trang tái sử dụng (từ trace) ở mục 6.
5. Chất lượng mã, git, `time.txt`	0.5	Xuất sắc	Mã theo phong cách xv6, có chú thích, `make fmt` không đổi tệp nhóm sửa; commit tách theo task của cả hai thành viên; `time.txt` = 7.
6. Báo cáo	1.0	Xuất sắc	Đủ các mục của đề: phân công, tự đánh giá, sáu câu gdb, tệp/trường/đường đi của `interpose`, giải thích tấn công.

Bảng 9. Nhóm tự đánh giá theo rubric của đề. Đây không phải điểm đã được giảng viên xác nhận.

9. Tài liệu tham khảo

[1] CS333 – Hệ điều hành, *Lab 2: System Calls* (đề bài, yêu cầu nộp bài và thang điểm), 2026.

[2] MIT 6.1810, *Lab: System calls*, Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/syscall.html

[3] R. Cox, M. F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system (RISC-V)*, chương 2, chương 3 (Hình 3-3) và mục 4.3–4.4. https://mit-pdos.github.io/xv6-riscv-book/

[4] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture* – thanh ghi `sstatus`, mã `scause`. https://github.com/riscv/riscv-isa-manual

[5] Mã nguồn xv6-labs-2026, nhánh `syscall`: `kernel/{syscall,trap,proc,exec,vm,kalloc}.c`, `kernel/trampoline.S`, `kernel/memlayout.h`, `user/{init,umalloc,sandbox,sbtest,secret}.c`. Bài làm của nhóm: commit `e969443` trên nhánh `syscall` (nội dung mã có trong `lab.zip`). Trace và log kiểm chứng: `docs/verification/` trong repo của nhóm.

[6] Công cụ AI. Claude Code: hỗ trợ tự động hóa và ghi lại phiên gdb, cài đặt và kiểm thử Task 3.2, phân tích đường đi của trang nhớ ở Task 3.4 (đo lại trace ngày 25/10), chụp ảnh minh chứng, soạn và chỉnh sửa báo cáo theo nhận xét rà soát. Codex: hỗ trợ rà soát mã và báo cáo, viết Task 3.3–3.4, viết test bổ sung và chạy kiểm chứng GDB/QEMU. Nhóm chịu trách nhiệm về toàn bộ nội dung bài nộp.